

Title: *Communities R Us*

Team Members: Mitchell Deevers, Jacob Wingstrom, Kayvon Pajouyan

Motivation:

We wanted a project that is simple at its core, but can be easily expanded upon. Given this fact our group cycled through a few ideas before we eventually decided a Reddit-like clone would be the best app to build for this project. The main idea of our app is that a user can log in/sign up into their account, create/join a community, and make a post in the communities they created/joined. With this base skeleton of the app, we can scale further, adding features such as up and down voting, etc. The app allows us to get an MVP that would be deployable given the time frame of the project, while still allowing for the opportunity to add many more features if we have time. We also believed this application would allow us to utilize all the skills we learned over this course, including an HTML frontend that will require CSS styling, a complex backend managed by Node.js with an Express framework, and a MongoDB backend that needs to manage all the items we need, such as users, posts, and communities.

Modules Overview:

1. Users:

The users module allows for the creation of new user accounts and their insertion into the database, and the login and session assignment using existing accounts. The session's user details are attached to post and community documents the user interacts with. Crypto is utilized in account registration and login, where passwords are stored in the db only after being hashed. Login inputs are hashed using the same exact hashing parameters and can then be compared to the stored credentials. On logout, the user details in the session are destroyed, and previously navigated pages are no longer routable.

2. Communities:

The communities module allows for authenticated users to search, create and open community posts. The searching is done purely on the front end using the filter function built into JavaScript. A user is able to create a community by pressing the button and inputting a name and description. This then reloads the communities showing the newest at the top. Then a user is able to click on any community and it will open the posts page for that community.

3. Posts:

The post module allows for authenticated users to interact with content within communities. Once a user has joined a community, they can navigate to that community's page and create a new post by providing a title and content (text and/or image). Upon submission of a post, the post is saved to the MongoDB collection with username, timestamp, content, and associated community. The post then appears on the community's feed where people can view it. Users can also reply to a community post they like. Image uploads are handled via multipart form data and stored server-side, with the file path saved in the post document so the image can be served back to client on demand. Users can also look at their profile and see all of their posts they have ever posted.

Functionalities and Workflows:

- **Functionality**
 1. User registration and login
 2. Create and join communities
 3. Search for communities by name
 4. Create posts within communities
 5. View all the posts within a community
 6. Reply to posts
- **Workflows**
 - **User Authentication**
 - **Step 1:** User navigates to the sign in/sign up page
 - **Step 2:** For registration, the user enters a username and password. The server salts and hashes the password before storing it.
 - **Step 3:** For login, the user submits a password and user name. If the password matches with the username in the backend the session is populated with the user details.
 - **Step 4:** On success, the session is created and the user is redirected to the community page
 - **Step 5:** On logout, the session is destroyed server-side and the user is redirected to the login page.
 - **Creating/Joining a Community**
 - **Step 6:** Authenticated user can browse the community page. Server fetches and displays all community documents from MongoDB, newest first.
 - **Step 7:** User clicks create community, enters a name, description, and then submits. The server then validates the input and inserts a new community document into the database.
 - **Step 8:** The new community appears at the top of the list. Any user can then search for it by name using front-end filter

- **Step 9:** A user clicks on any community to open its posts page. Navigating to a community's post page gives the user access to create a post within it.
- **Creating and Interacting with Posts**
 - **Step 10:** User navigates to a community page. All posts for that community are displayed in reverse-chronological order.
 - **Step 11:** User clicks 'New Post', enters a title, content, and submits. The server saves the post document with username, timestamp, community ID, and content.
 - **Step 12:** The new post appears at the top of the feed. Other users who are viewing the same community will see the new post on their next load.
 - **Step 13:** Any authenticator user can expand a post to view replies and add their own. Reply documents are stored with a reference to parent post id, allowing for a threaded display on the frontend.

Technical Details:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js with Express framework
- **Database:** MongoDB-Used to store users, communities, and posts as collections.
- **Routing:** The Express router organizes endpoints into separate route files for user authentication, communities, and posts. (Professor approval was received)
- **Session handling:** Session management is handled through Express sessions middleware, so as not to hassle over browser storage management and potential security compromises.
- **File Structure:** Our project is structured with modularity and scalability in mind. We prioritized the separation of responsibilities early on, writing no functions that mix direct database access with express endpoints. Our more sensitive source code is also separate from our static files.
- **Server:** Node runtime with Express, runs locally on port 8080